

Fall 2026 Exploratory Data Analysis with R

Lecture 10: Coding Review and Functional Programming

Brian Robertson
Federal Reserve Bank of Atlanta
2026-11-06

01

Recap & Agenda

01 Recap & Agenda

02 Code Review

03 Writing Functions

04 Functional Programming

05 Recap

Quick Recap: Last Week

- **Regression II & III** — building multiple regression models and interpreting their coefficients

Today: a shift back from statistics to programming. We'll **review code** we've all written this semester against the course's style conventions, then formalize a tool for automating repeated tasks — **functional programming**.

Goals for Today

- Review real code against the course's style conventions, and recognize a few common bug patterns
- Write your own function — name, arguments, body, and return value
- Recognize when repeated, copy-pasted code should become a function call that runs once per input
- Use `purrr::map()` and its typed variants to apply a function across many inputs, instead of base R's `sapply()/lapply()`

No new homework or project update this week — today is lighter-weight, focused on practice rather than a new deliverable.

02

Code Review

- 01 Recap & Agenda
- 02 Code Review**
- 03 Writing Functions
- 04 Functional Programming
- 05 Recap

Why Review Code?

- Code can run without errors and still be **wrong** — review catches mistakes testing alone might miss
- Code can be correct and still be **hard to read** — review catches style problems before they become someone else's bug to debug
- You will read far more code than you write — your adviser's, a classmate's, your own from three weeks ago

Reviewing someone else's function doesn't require knowing how to write one yet — it just requires reading carefully and checking it against a few consistent rules.

This Course's Style Checklist

A few rules we've been following all semester — review code against these before you check it for anything else:

- **Never use `$` to pull a column off a data frame.** Use `pull()` inside a pipe instead
- **Reach for a tidyverse function before a base R one**, when both exist — `pull()` over `$`, `case_when()`/
`if_else()` over nested base `ifelse()`, `map()` over `sapply()`/`lapply()` (more on that later today)
- Name things for what they hold, not for their type (`loan_df`, not `data1`)
- A function should do one job and return one clear thing

None of this changes *what* the code computes — it changes whether the next person (including future you) can trust it at a glance.

Review 1: \$ on a Data Frame

A classmate wrote this function to find the average monthly payment on a loan.

```
loan_2013 <- read_csv("data/loan_2013.csv", show_col_types = FALSE)

average_payment <- function(loan_df) {
  mean(loan_df$payment)
}

average_payment(loan_2013)
```

```
[1] 35.20417
```

- It returns the right number — so what would you flag?

Fix: `pull()` instead of `$`

```
average_payment <- function(loan_df) {  
  loan_df %>%  
    pull(payment) %>%  
    mean()  
}  
  
average_payment(loan_2013)
```

```
[1] 35.20417
```

- Same answer, same function — but it reads as one pipeline instead of mixing pipe style with base indexing

Review 2: Order of Conditions

Another classmate wrote a `grade()` function using `case_when()` — but swapped two lines while editing it.

```
grade <- function(score) {  
  case_when(  
    score >= 90 ~ "A",  
    score >= 70 ~ "C",  
    score >= 80 ~ "B",  
    score >= 60 ~ "D",  
    TRUE ~ "F"  
  )  
}
```

```
grade(85)
```

```
[1] "C"
```

- An 85 should be a B. Why did this return "C"?
- `case_when()` checks conditions **in order** and stops at the first **TRUE** — just like a chain of `if/else if` statements. An 85 is also `>= 70`, so the C condition fires first and the B condition never gets checked

Fix: order conditions from most to least restrictive

```
grade <- function(score) {  
  case_when(  
    score >= 90 ~ "A",  
    score >= 80 ~ "B",  
    score >= 70 ~ "C",  
    score >= 60 ~ "D",  
    TRUE ~ "F"  
  )  
}
```

```
grade(85)
```

```
[1] "B"
```

```
grade(93)
```

```
[1] "A"
```

Review 3: A Hidden Assumption

A friend wrote a function to decode a secret message by reversing a string, and a vector of reversed messages to test it on.

```
reverse_string <- function(string) {  
  string %>%  
    strsplit(split = NULL) %>%  
    .[[1]] %>%  
    rev() %>%  
    paste0(collapse = "")  
}  
  
cipher <- c(".rehpic eht si sihT", ",boj dooG",  
            "!rehpic eht dekarc uoy",  
            ".lufpleh repus eb nac gnimmargorp lanoitcnuF")  
  
reverse_string(cipher[1])
```

```
[1] "This is the cipher."
```

- Looks right on a single message. What happens on the whole vector?

```
reverse_string(cipher)
```

```
[1] "This is the cipher."
```

- No error — but three of the four messages vanished. `strsplit()` returns one list element **per string**, and `.[[1]]` always grabs just the first one. The function quietly assumes its input is a single string, and nothing warns you when that assumption breaks
- This is the most dangerous kind of bug: wrong, but silent. We'll come back to this exact function once we have the right tool to fix it

Breakout: Code Review

- Open `lecture10_inclass_exercise.R`
- Work through the Code Review section — each function has a style problem, a logic bug, or both
- For each one: flag what you'd leave as a comment in a real review, then fix it
- 10 minutes

03

Writing Functions

- 01 Recap & Agenda
- 02 Code Review
- 03 Writing Functions**
- 04 Functional Programming
- 05 Recap

Why Write Your Own Function?

Back on Day 1 we said functions come from three sources: base R, packages, and **user-defined** — functions we write ourselves. We've only used the first two so far.

- **One place to fix a mistake**, instead of every copy-pasted spot it appears
- **A clear name** for what a block of code does, instead of re-reading the whole block each time
- **Fewer chances to typo** — write the logic once, call it as many times as you need

Anatomy of a Function

Every function has four parts:

1. A **name** — what you'll call it by
2. **Arguments** — the inputs it needs, listed in `function(...)`
3. A **body** — the code between `{ }` that does the work
4. A **return value** — the one object the function hands back

Anatomy of a Function

```
function_name <- function(argument1, argument2) {  
  # body: code using argument1 and argument2  
  return(result)  
}
```

- Same shape as a `for` loop or an `if/else` statement — parentheses hold the setup, braces hold the code
- `return()` is technically optional — R returns the last expression evaluated in the body if you leave it off — but write it explicitly while you're learning. It makes the one thing you're handing back unambiguous

From a Script to a Function

Suppose we need to present fractional numbers as rounded percentages.

```
x <- c(0.458, 0.667, 0.831)

x_times_100 <- x * 100
rounded <- round(x_times_100, digits = 1)
result <- paste0(rounded, "%")

result
```

```
[1] "45.8%" "66.7%" "83.1%"
```

- This works — but only for the object named `x`. To do it again for a different vector, we'd copy-paste the same three lines

From a Script to a Function

Wrap the same steps in a function, and replace `x` with an **argument name**.

```
add_percent <- function(x) {  
  rounded <- round(x * 100, digits = 1)  
  paste0(rounded, "%")  
}  
  
add_percent(x = c(0.458, 0.667, 0.831))
```

```
[1] "45.8%" "66.7%" "83.1%"
```

```
add_percent(x = c(0.05, 0.5, 0.95))
```

```
[1] "5%" "50%" "95%"
```

- Note `add_percent()` works on a vector of any length without changes — basic arithmetic and `paste0()` are already **vectorized**, so one call handles every element. Not every function gets this for free, as we saw with `reverse_string()` a moment ago

Default Arguments

An argument can have a **default value**, used whenever the caller doesn't supply one. Recall `grade()` from code review, now fixed:

```
grade <- function(score) {  
  case_when(  
    score >= 90 ~ "A",  
    score >= 80 ~ "B",  
    score >= 70 ~ "C",  
    score >= 60 ~ "D",  
    TRUE ~ "F"  
  )  
}
```

Let's build a second function **on top of it** to account for extra credit:

```
grade_with_extra <- function(score, extra = 0) {  
  grade(score + extra)  
}
```

```
grade_with_extra(score = 69.99, extra = 25)
```

```
[1] "A"
```

```
grade_with_extra(score = 88)
```

```
[1] "B"
```

- `extra = 0` is the default — omit it and nothing changes
- `grade_with_extra()` calls `grade()` inside its body. Building small functions out of other small functions is good practice: each one is easier to read, test, and fix on its own

Breakout: Converting Temperatures

Write three functions:

From	To	Formula
Fahrenheit	Kelvin	$K = (F - 32) \times \frac{5}{9} + 273.15$
Kelvin	Celsius	$C = K - 273.15$
Fahrenheit	Celsius	???

1. Write `F_to_K()` and `K_to_C()` directly from the formulas above
2. Write `F_to_C()` **by calling the two functions you just wrote** — don't re-derive the formula
3. Open `lecture10_inclass_exercise.R` and check your answers against body temperature (98.6°F) and freezing (32°F)

Breakout: Solution

```
F_to_K <- function(degrees_f) {  
  ((degrees_f - 32) * (5 / 9)) + 273.15  
}  
  
K_to_C <- function(degrees_k) {  
  degrees_k - 273.15  
}  
  
F_to_C <- function(degrees_f) {  
  K_to_C(F_to_K(degrees_f))  
}  
  
F_to_C(degrees_f = 98.6)
```

```
[1] 37
```

```
F_to_C(degrees_f = 32)
```

```
[1] 0
```

04

Functional Programming

- 01 Recap & Agenda
- 02 Code Review
- 03 Writing Functions
- 04 Functional Programming**
- 05 Recap

When One Function Call Isn't Enough

Suppose we have loan records for four separate students, one CSV per loan, and we want the total interest paid on each.

```
loan_2013 <- read_csv("data/loan_2013.csv")
interest_2013 <- sum(pull(loan_2013, interest))

loan_2014 <- read_csv("data/loan_2014.csv")
interest_2014 <- sum(pull(loan_2014, interest))

loan_2015 <- read_csv("data/loan_2015.csv")
interest_2015 <- sum(pull(loan_2015, interest))

loan_2016 <- read_csv("data/loan_2016.csv")
interest_2016 <- sum(pull(loan_2016, interest))
```

- Four files, same two steps, copy-pasted four times — the exact pattern that called for a `for` loop a few weeks ago

- We already know the fix for the *repetition*: write a function. What's new today is **a tool built specifically to call a function once per element of a set**

Functional Programming: The Idea

- **Functional programming (FP)** means describing a task as “apply this function to every element of that set,” instead of writing out the steps for each element by hand
- The function already exists — in this case, “read a file and total its interest column.” We just need to run it four times, once per filename
- R has two toolkits for this: base R’s `apply` family, and the tidyverse’s `purrr::map()` family

Step 1: Write the Function Once

```
loan_files <- c("data/loan_2013.csv", "data/loan_2014.csv",  
               "data/loan_2015.csv", "data/loan_2016.csv")  
  
total_interest_paid <- function(file) {  
  file %>%  
    read_csv(show_col_types = FALSE) %>%  
    pull(interest) %>%  
    sum()  
}  
  
total_interest_paid(loan_files[1])
```

```
[1] 724.51
```

- One file in, one number out — exactly what we copy-pasted four times a moment ago, now written once

Step 2: Base R's `apply` Family

`sapply()` and `lapply()` call a function once per element of a vector or list. You'll see these constantly in other people's R code.

```
sapply(loan_files, total_interest_paid)
```

```
data/loan_2013.csv data/loan_2014.csv data/loan_2015.csv data/loan_2016.csv  
724.51 1138.07 1273.39 1107.13
```

```
lapply(loan_files, total_interest_paid)
```

```
[[1]]  
[1] 724.51
```

```
[[2]]
[1] 1138.07
```

```
[[3]]
[1] 1273.39
```

```
[[4]]
[1] 1107.13
```

- Same function, same inputs, two different output shapes — `sapply()` tried to simplify its result into a named vector; `lapply()` always hands back a list, one element per input

Meet `purrr::map()`

`sapply()`'s “try to simplify” behavior is convenient right up until it silently simplifies into the wrong shape. The tidyverse's `purrr` package replaces it with **one function per output type**:

- `map()` — always returns a list, like `lapply()`
- `map_dbl()` — always returns a vector of doubles, or errors
- `map_chr()` — always returns a vector of character strings, or errors
- `map_int()`, `map_lgl()` — a vector of integers, or of `TRUE/FALSE`

The “or errors” part is the point: if the function doesn't return what you promised, you find out immediately — not three steps later when something downstream breaks.

Why the Type Promise Matters

Suppose a function has a bug and returns the *range* of interest paid (two numbers) instead of the total (one number).

```
buggy_interest_stats <- function(file) {  
  file %>%  
    read_csv(show_col_types = FALSE) %>%  
    pull(interest) %>%  
    range()  
}  
  
sapply(loan_files, buggy_interest_stats)
```

	data/loan_2013.csv	data/loan_2014.csv	data/loan_2015.csv
[1,]	0.11	0.18	0.20
[2,]	11.26	17.48	19.66

```
data/loan_2016.csv  
[1,]          0.17  
[2,]         17.23
```

- No error — `sapply()` quietly reshapes the result into a 2-row matrix. Easy to miss in a long script

```
map_dbl(loan_files, buggy_interest_stats)
```

```
Error in `map_dbl()`:  
i In index: 1.  
Caused by error:  
! Result must be length 1, not 2.
```

- `map_dbl()` refuses, with a message pointing at exactly which input broke the promise

Putting `map_dbl()` to Work

Back to `total_interest_paid()`, which keeps its promise — one number per file. Name the files first, and the result comes back labeled.

```
named_files <- set_names(loan_files, c("2013", "2014", "2015", "2016"))  
  
map_dbl(named_files, total_interest_paid)
```

```
  2013    2014    2015    2016  
724.51 1138.07 1273.39 1107.13
```

- Same function we wrote for the base R example — only the *tool that calls it* changed
- Going forward, reach for `map()/map_dbl()/map_chr()` over `sapply()/lapply()`. You'll see the base R versions in code you read, especially older code, but write new code with `purrr`

Back to the Cipher

Recall `reverse_string()` from code review — correct on one string, silently wrong on a vector of four.

```
map_chr(cipher, reverse_string)
```

```
[1] "This is the cipher."  
[2] "Good job,"  
[3] "you cracked the cipher!"  
[4] "Functional programming can be super helpful."
```

- `map_chr()` calls `reverse_string()` once per element of `cipher` and stitches the four results back into a character vector — no change to `reverse_string()` itself
- The bug in Review 3 was never really inside `reverse_string()`. It was calling a function built for *one* input on a *vector* of them, by hand, instead of handing that job to a tool built for exactly this

Breakout: Functional Programming

- Open `lecture10_inclass_exercise.R`
- Work through the Functional Programming section:
 1. Use `map_dbl()` to get the total interest paid on all four loans at once
 2. Use `sapply()` on the same function and inputs — compare the two outputs
 3. **Bonus:** write a function that returns a loan's total *principal* paid, and `map_dbl()` it across all four files
- 10 minutes, then we'll wrap up

05

Recap

- 01 Recap & Agenda
- 02 Code Review
- 03 Writing Functions
- 04 Functional Programming
- 05 Recap**

Recap

- **Code review** checks correctness and style — watch for `$` on a data frame, condition order in `case_when()`, and functions that silently assume a single-input shape
- A **function** has a name, arguments, a body, and a return value — write it once, call it as many times as you need, and build complex functions out of simple ones
- **Functional programming** means handing a function to a tool that calls it once per element, instead of copy-pasting the call
- `purrr::map()` and its typed variants (`map_dbl()`, `map_chr()`, ...) are the tidyverse's answer to base R's `sapply()/lapply()` — use them going forward

Functions Introduced Today

- `pull()` — extract a column from a data frame without `$`
- `case_when()` — recall that conditions are checked in order, first match wins
- `set_names()` — attach names to a vector before mapping over it
- `sapply()`, `lapply()` — base R's function-per-element tools
- `map()`, `map_dbl()`, `map_chr()` — `purrr`'s typed function-per-element tools

Documentation & Further Reading

- `?function`, `?return` — writing your own functions
- `?purrr::map`, `?purrr::map_dbl` — the `map()` family and its typed variants
- `?sapply`, `?lapply` — base R's function-per-element tools
- <https://adv-r.hadley.nz/functionals.html> — a deeper tour of functional programming in R

Coming up: correlation vs. causation, and the first draft of your project.